SOFTWARE ENGINEERING LAB 4

Vibe Coding – Chat History

Name	Anand S
USN	PES1UG24AM341
Project Name	Fruit Ninja – Python/Pygame
Lab	Software Engineering Lab – Lab 4
Repository	SE_Lab4/41_fruit-ninja

This document contains the questions/prompts used during the Vibe Coding process for the four assigned tasks, along with the corresponding implementation direction and testing outcome.

TASK 1 – REFINE COLLISION DETECTION

USER – QUESTION / PROMPT

Improve the collision detection in the Fruit Ninja game so that fruit slicing works reliably, especially when the player performs fast mouse swipes. Use the line segment between the previous mouse position and the current mouse position for collision detection instead of checking only the current mouse position. Also add a small collision margin around the fruit so that fast swipes are detected reliably. Do not change unrelated gameplay features.

LLM – IMPLEMENTATION RESPONSE / DIRECTION

The collision detection was improved to check the line segment between the previous mouse position and the current mouse position instead of checking only the current mouse position. An additional collision margin was added around the fruit to improve detection reliability during fast swipes.

USER – TESTING / RESULT

The game was tested by performing fast mouse swipes across fruits. Slicing was detected reliably, including when the mouse moved quickly between frames. Task 1 was confirmed working.

TASK 2 – IMPLEMENT GAME OVER SCREEN

USER – QUESTION / PROMPT

Implement a proper Game Over screen for the Fruit Ninja game. When the player loses, stop the normal gameplay and display a clear Game Over message/state to the player. The Game Over state should integrate cleanly with the existing game flow and should not affect unrelated functionality.

LLM – IMPLEMENTATION RESPONSE / DIRECTION

A Game Over state and screen were added. When a Game Over condition occurs, normal gameplay stops and the player sees a clear Game Over message. This state integrates with the game flow and supports the replay/difficulty selection flow added in Task 3.

USER – TESTING / RESULT

The game was played until a Game Over condition occurred. The Game Over screen appeared and gameplay stopped as expected. Task 2 was confirmed working.

TASK 3 – ADD REPLAY AND DIFFICULTY SELECTION

USER – QUESTION / PROMPT

Add replay functionality and difficulty selection to the Fruit Ninja game. After Game Over, provide options for Easy, Medium, Hard, and Exit. Make the difficulty levels meaningfully different by changing gameplay parameters such as fruit spawn interval, bomb probability, and object speed. Allow the player to start a new game without restarting the application. Keep the existing gameplay and collision improvements working correctly.

LLM – IMPLEMENTATION RESPONSE / DIRECTION

Replay and difficulty selection were added to the game flow. The available options are Easy, Medium, Hard, and Exit. Difficulty settings adjust gameplay parameters such as fruit spawning interval, bomb probability, and object speed. The player can start a new game without restarting the application. Gameplay and collision-related issues encountered during implementation were debugged.

USER – TESTING / RESULT

The replay and difficulty selection flow was tested. Easy, Medium, and Hard could be selected, and replay started a new game without restarting the application. Exit was also checked. Task 3 was confirmed working.

TASK 4 – ADD SOUND EFFECTS

USER – QUESTION / PROMPT

Add sound effects to the Fruit Ninja game for the major gameplay events. Implement a sound management module and add separate audio feedback for fruit slicing, bomb explosion, and Game Over. Ensure the sounds are triggered at the correct events and do not break the existing gameplay, replay, difficulty selection, or collision detection features.

LLM – IMPLEMENTATION RESPONSE / DIRECTION

A sound management module, game/sound_manager.py, was added. Sound effects were implemented for fruit slicing, bomb explosion, and Game Over. The sounds are triggered by their corresponding gameplay events. The bomb sound behaviour was checked and corrected during testing.

USER – TESTING / RESULT

Fruit slicing, bomb explosion, and Game Over sounds were tested. Each sound played at the appropriate event, and the earlier gameplay features continued to work. Task 4 was confirmed working.

FINAL PROJECT STATUS

Task	Feature	Status
Task 1	Refine Collision Detection	Completed
Task 2	Game Over Screen	Completed
Task 3	Replay + Easy/Medium/Hard Difficulty	Completed
Task 4	Fruit / Bomb / Game Over Sound Effects	Completed

Final deliverables completed: before-change video, after-change video, updated source code pushed to the personal GitHub repository, and this Vibe Coding chat history document.